Compression Is Intelligence: Rethinking Neural Network Generalization

Chao Zheng
Independent Researcher

Version 1.1 - September 20, 2026

Abstract

A finite training set usually admits many distinct functions: they give the same answers on the training samples but different predictions on unseen inputs. Random initialization already gives neural networks a highly nonuniform preference over these functions, favoring simpler ones. This is an important starting point for explaining generalization, but it does not fully describe training. On enumerable input domains, we record a network's discrete predictions on every possible input and treat this complete prediction table as a function. We find that actual training continues to change the distribution of functions on unseen inputs even after every training sample has been classified correctly. The resulting distribution also differs substantially from one obtained by deleting, in a single step, all functions in the initialization distribution that disagree with the training labels.

To separate this phenomenon from optimizer dynamics, we fix the network, data encoding, and parameter-sampling measure, perform no gradient training, and examine only parameters whose loss is below a sequence of thresholds. As the threshold is lowered, the parameter probabilities associated with different complete functions do not decrease in the same proportion. A function that was initially more common can be overtaken by another, and two probability curves can cross even at very low loss. We therefore define a Neural Kolmogorov complexity curve: for a complete target, we record the parameter probability of reaching each loss threshold and take its negative logarithm. This quantity depends on the particular network setup and is not machine-independent Kolmogorov complexity. In our experiments, curves measured in advance predict three kinds of outcomes that were not used to define them: how many training samples are typically required to recover a rule completely, which candidate label for an unlabeled example is more probable, and the population frequencies of complete functions produced by many independent training runs of the same small network.

The results also show that three questions must be distinguished: how much low-loss parameter volume a complete target function has on its own; what fraction it occupies among all functions that fit a given finite training set equally well; and which functions are actually obtained after training. The data determine which incorrect answers have not yet been ruled out. The architecture and parameter-sampling measure determine which computations are easy to implement. The loss threshold determines how far the training error must be reduced. The optimizer and training history determine which parameters can actually be reached. This division of roles provides a common language for direct learning, grokking, overfitting, and memorization of noise.

1. Empirical Starting Point: Neural Networks Do More Than Memorize Samples

This study did not begin with a small task designed specifically for grokking. Our earlier exploration systematically asked whether neural networks could learn rule-based tasks. We experimented with cellular automata, arithmetic with permuted symbols, trapping rain water, mazes, image transformations, and many other program-generated mappings. MLPs, CNNs, RNNs, and Transformers all recovered fixed-size deterministic rules from training sets far smaller than their complete input spaces. Addition remained learnable after digit symbols and positions were permuted, while MNIST-CA placed pattern recognition and rule learning inside the same end-to-end network. These results do not prove that the networks used human-like program logic internally. They do show that ordinary neural networks can learn rule execution and pattern recognition together through the same training loss.

These experiments also showed that rule learning does not always take the form of grokking. With enough data, training and validation performance often improve together from an early stage, a regime we call direct learning. Near the amount of data needed to identify a rule, a network may first fit the training set and improve on unseen inputs only after a long delay, eventually reaching complete generalization. With still less data, it may remain indefinitely at an incorrect rule that exploits gaps in the training set. Rule-duel and OOD rule-generalization experiments further showed that, after training accuracy reaches 100%, networks continue to change their predictions on unseen inputs and continue moving toward functions that are simpler under a complexity comparison specified in advance.

The question therefore shifts from whether a network has enough capacity to learn a rule to which function training selects from the many functions consistent with the training set. Let the set of all possible inputs be $\mathcal{X}$, of which the finite training set D covers only a subset. We call all class labels produced by a model over $\mathcal{X}$ its complete function. Many complete functions can answer every example in D correctly while disagreeing on unseen inputs; they are different completions of the training data. An overparameterized network can represent many such completions, so expressivity alone cannot tell us which one will be selected.

Previous work provides a first answer. Because many different parameters can realize the same function, and random parameters do not produce all functions uniformly, many network architectures generate structurally simple functions more often than complex ones (Dingle et al., 2018; Valle-Perez et al., 2019). The frequency of functions induced by random initial parameters is commonly called the initialization function prior. Mingard et al. (2021, 2025) further showed that conditioning this prior on consistency with the training labels can explain a substantial part of the function distribution produced by SGD. Wilson (2025) describes this property, which does not forbid complex solutions but assigns different probabilities to different solutions, as a soft inductive bias. We accept this result but ask a further question: if training merely filters incorrect answers out of the initialization distribution, why does the function distribution keep changing after every training sample is already correct?

2. Networks Continue to Change Unseen Predictions After Classifying the Entire Training Set Correctly

Neural networks output continuous logits, the real-valued scores that precede class labels. Even when two networks classify the same example correctly, their scores for the correct class can differ greatly. We use *hard fit* to mean that every training sample has been classified correctly. At this point, training loss can still decrease, the margin between the correct class and the decision boundary can still grow, and unseen inputs can still change class. A training

accuracy of 100% is therefore not a natural endpoint of continuous optimization (Soudry et al., 2018).

The simplest initial model can be written as follows. First estimate the probability $P_{\text{init}}(f)$ that random initialization produces function f. Then remove every function inconsistent with the training labels and renormalize:

$$Q_{\text{hard}}(f \mid D) \propto P_{\text{init}}(f)\mathbf{1}[f \models D].$$

This procedure performs no gradient descent. It merely removes, from the functions produced by random initialization, those that answer at least one training example incorrectly. If this explanation were sufficient, the relative proportions among the surviving functions should not continue to change systematically after hard fit.

Experiment E01 directly rejects this prediction. In a 3-bit Boolean task there are only eight possible inputs, so each network's answers on the entire domain can be recorded exactly. More than one million untrained networks establish the initialization frequencies of all observed functions. By the time actual Adam training first classifies the whole training set correctly, its function frequencies already differ markedly from the prediction above. The discrepancy grows with continued training, and the most common function can change. In E03, we first filter the initialization distribution to obtain networks that already answer all training labels correctly, so their function proportions agree with the one-step conditioning model. After only one gradient update, most of these networks change their answers on unseen inputs. E04 reproduces the same phenomenon in a deeper, previously studied Boolean setup.

E07 provides a more direct view by recording every training trajectory. Among 1,024 independent runs, 1,019 change their complete function at least once after hard fit. As training continues, an increasing number of initializations converge to a small number of common functions. Parameter changes after hard fit are therefore not functionally irrelevant: while training loss and logits continue to change, the network's class labels on unseen inputs continue to change as well.

Nor do all initializations settle on the same incorrect algorithm before grokking. E27 records every run's answers on all unseen inputs in a Mod97 task and defines *Agreement* as the fraction of answers shared by a pair of runs. At hard fit, Agreement remains near its random baseline. Test performance and Agreement then rise together, mainly because more runs jointly recover the target while the remaining errors continue to differ across runs. Before grokking, there is therefore a population of distinct functions that all fit the training set, rather than one common incorrect program. During grokking, more and more initializations reach the same target function; they do not all jump from one shared incorrect program to the correct one.

These results establish that the function is changing, but not yet that it changes toward simplicity. To ask whether the selection comes from the loss landscape itself, we must temporarily remove the optimizer.

3. Continuous Loss Is Itself a Scale of Function Selection

Fix a network architecture, input-output encoding, and parameter probability measure μ. Instead of running Adam or SGD, consider only parameters whose training loss is no greater than a threshold ϵ:

$$A_D(\epsilon) = \{\theta : L_D(\theta) \leq \epsilon\}.$$

Within $A_D(\epsilon)$, we can measure how much parameter probability is assigned to each complete function. We call the resulting distribution, obtained without running training and conditioned only on loss, the *static function distribution*. Here, a function's parameter volume means probability relative to the prespecified sampling measure μ; it is not a coordinate-invariant geometric volume. When ordinary random sampling almost never finds low-loss parameters, sequential Monte Carlo (SMC) can lower the permitted loss gradually while repeatedly retaining and perturbing better parameters. Hamiltonian Monte Carlo (HMC) instead uses gradients to propose longer moves in parameter space and applies an accept-or-reject step to preserve its target distribution.

The simplest hypothesis is that, as the loss threshold decreases, every function consistent with the training labels becomes rarer by the same factor, leaving their relative frequencies unchanged. E06 examines millions of untrained networks while continuously lowering training loss under the constraint that every training label remains correct. Function frequencies still change by orders of magnitude, and a symmetric function that is initially rare can become the most common one. Thus, even with no actual training, continuous changes in loss reweight the parameter probability assigned to different functions.

This change cannot be labeled "simplicity" only after observing which function wins. E10 specifies pairs of functions in advance. Both members of a pair satisfy the same training labels; one uses a shared rule that can be reused across inputs, while the other adds several independent exceptions. Across twelve reliable comparisons, the relative mass of the shared rule increases every time the loss is lowered. This supports a limited mechanism: if an implementation must coordinate more independent residual directions, then simultaneously tightening the allowed error in every direction will usually make its parameter probability shrink faster. Shared intermediate computation allows multiple examples to reuse the same adjustments and can therefore lose probability more slowly.

E08 tests the same explanation from the side of network structure. When two output tasks genuinely contain the same initial steps of a cellular-automaton computation, making them share the corresponding intermediate network lowers the attainable loss more effectively than using two fully separate branches with the same total parameter count. When the tasks share no computation, the shared network has no stable advantage. The benefit grows when the common prefix is longer and capacity is tighter. This experiment does not yet yield a quantitative formula, but it directly supports the idea that reusing a computation reduces the number of parameter directions that must be adjusted independently.

But the "true rule" in the researcher's eyes has no privileged status. E12 deliberately leaves a gap in the training set of an AND task, allowing another, incorrect rule to answer every training sample correctly. Replacing only one sample with an example that exposes the incorrect rule greatly increases the fraction of low-loss parameters that implement AND. E13 finds the same direction of change in actual training. The network receives only the observed samples and the loss; it does not know which generating rule the researcher wants it to recover. From the network's perspective, there is no a priori privileged true rule. A training set may have been sampled from a well-defined distribution, or even generated by a program, but that external provenance does not by itself privilege the generating distribution within the optimization problem. Although higher test accuracy is commonly described as better generalization, the only objective available to the neural network is to reduce training loss. Everything beyond the training set is "out of distribution" in the special sense used here, and generalization is only a by-product. This use of "out of distribution" differs from the conventional term OOD; it emphasizes that the network can see only the training set. Neural-network optimization is centered on the training set rather than on a presumed, a

priori true rule, and its direct concern is reducing training loss. Achieving "generalization" is therefore not the network's own purpose, but a human expectation and evaluation objective. Determining which functions are simpler for a neural network requires considering both which incorrect answers the training set has not excluded and which functions the current network can express easily. Simplicity is relative to the training problem, not evidence that the network is moving toward a mysterious, preordained true function. This issue is especially visible when data are scarce, where it can be difficult to predict which function the training set will make the network prefer.

The most common function also need not remain the same as loss decreases. E14 constructs a task in which a control bit selects between two rules, and then reduces the contribution of a small number of conflicting samples to the total loss. When relatively high loss is allowed, implementing only the rule required by the majority of samples removes most of the error. At lower loss, the model must also handle the rare conflicting examples and implement the complete composite rule. Changing the weights of those rare examples changes the loss at which the two functions overtake each other. Lowering loss does not merely amplify a fixed winner; it progressively raises the precision to which every training example must be satisfied.

This gives an interpretation compatible with minimum description length (MDL). Lowering training loss both eliminates functions that truly violate the training labels and penalizes implementations that require many nonreusable degrees of freedom to reduce all residuals simultaneously. The first effect enforces the known data; the second suppresses unnecessary additional structure beyond the training set. This is not a theorem that functions humans consider simple must win. Rankings depend on the architecture, encoding, samples, and required precision, and relative rankings can cross.

4. Neural Kolmogorov Complexity Is a Curve, Not a Permanent Difficulty Score

To measure how difficult a target is to encounter in a given network, we temporarily provide its complete truth table on a finite input domain, so that every parameter setting faces the same unique target. Let Π denote the network architecture, parameterization, data encoding, parameter-sampling distribution, and per-sample loss. The parameter probability of realizing a complete function f at precision ϵ is

$$V_{\Pi}^{\text{full}}(f;\epsilon) = \mu_{\Pi}\{\theta : L_{D_{\text{full}}(f)}(\theta) \le \epsilon\},$$

and we define

$$K_{\Pi}^{N,\text{full}}(f;\epsilon) = -\log_2 V_{\Pi}^{\text{full}}(f;\epsilon).$$

The full curve traced by $K^{N,\text{full}}$ as ϵ changes is the **Neural Kolmogorov complexity curve** in this paper. It answers the question: "For this particular network, how difficult is it to encounter parameters that realize the complete target at each required precision when parameters are drawn from the specified distribution?" A probability of $1/1024$ corresponds to 10 bits. This number can be interpreted as a description length induced by the current network setup, but it is not Kolmogorov complexity relative to a universal Turing machine. Changing the network, initialization scale, or data encoding can change the curve. The fixed-training-set function-cell mass is denoted $\Omega_{\Pi,U,D}(f;\epsilon)$, or by its negative-log profile $K_{\Pi,U,D}^{N,\text{FD}}(f;\epsilon)$; it is a different object.

E19 measures a collection of rules from their complete 4-bit truth tables. As more input bits participate in a parity computation, the parameter probability of reaching the same loss threshold decreases. Majority functions and random truth tables can look similar when relatively high loss is allowed, yet differ by many orders of magnitude when lower loss is required. The function preference induced by network structure may therefore be weak under coarse fitting and become amplified only when the model must answer with greater confidence. Parity has a short human description but can still be difficult for an ordinary MLP to implement, so difficulty here is always relative to the chosen network.

The probability of a complete target is not the same as its probability of recovery from a finite training set. For fixed D, we instead measure the probability that parameters both satisfy $L_D \leq \epsilon$ and implement f over the complete domain:

$$\Omega_{\Pi,D}(f;\epsilon) = \mu_\Pi\{\theta : L_D(\theta) \leq \epsilon,\ h_\theta = f\}.$$

Only after dividing this quantity by the total probability of all functions satisfying the same training-loss condition do we obtain the fraction occupied by f under the current training set. Measuring a complete target asks how much low-loss parameter mass f has on its own. With only a finite training set, we must also account for the combined mass of every competing incorrect function.

When the complete-target loss event is guaranteed to lie inside the target hard cell, the two quantities are connected by conditional measure: complete-target mass equals the absolute mass of the target cell inside the parent training event, multiplied by the fraction of that cell that satisfies the additional continuous-precision requirement. At higher loss, or under different average-loss thresholds, complete-target mass cannot be identified directly with the fixed-training-set function-cell mass.

E20 further tests how these probabilities are related. At the same range of training loss, multiple complete functions usually remain consistent with the training data, and each function is implemented by many different parameter settings. Even when these parameters give the same binary answer on an unseen input, their logits can differ: some have only just crossed the classification boundary, while others support the class with a large margin. E20 first groups parameters by complete function and then progressively lowers the allowed loss on the unseen input. The number of parameters from each group that satisfies the new requirement decreases at a different rate, changing the groups' relative proportions. The experiment therefore checks the conditional-measure relation among complete-target mass, fixed-training-set function-cell mass, and within-cell continuous precision. These measurements obey ordinary conditional-probability identities, but they are not the same quantity.

E24 shows why the entire curve must be retained. A Parity4 rule and a rule obtained by flipping only one target label have one difficulty ordering when relatively high loss is allowed. At lower loss, their probability curves cross and the ordering reverses by many orders of magnitude. By the time of the crossing, both groups of networks already give every discrete answer specified by their respective targets. The change arises because the two parameter masses shrink at different rates as the models increase their output precision. There is therefore no loss-independent table of permanent function difficulty.

5. Can the Curve Predict Other Experiments?

Taking the negative logarithm of a probability is only a definition. The falsifiable claim is that this curve can predict other experiments before their training outcomes are observed. In E23, we first measure complete-truth-table curves for eight 8-bit rules and use a method fixed

before the experiment to extract a low-loss difficulty score from each curve. Only then do we run nearly ten thousand training trials across targets, sample sizes, and randomly drawn training sets. For every rule, we record how many training examples are required before the probability of complete recovery first reaches 50% and 90%. Among Parity1 through Parity4, the prespecified difficulty ordering agrees exactly with the sample-requirement ordering. Across all eight rules, the two rankings remain strongly correlated.

MUX3 provides an important counterexample. Its first input bit determines whether the output should copy the second or third bit. The complete MUX3 target has more low-loss parameter mass than Parity2, yet it requires more uniformly sampled examples to recover. On many inputs, the second and third bits happen to agree, so those examples cannot distinguish MUX3 from competing rules such as always copying the second bit. Changing only the input-sampling scheme so that conflicts between the second and third bits occur more often sharply reduces the sample requirement of MUX3, even below that of Parity2. A static comparison of the parameter probabilities left by fixed training sets, without running actual training, changes in the same direction. A target that is easy to implement is therefore not necessarily easy to identify from finite data: it may face strong competing functions that the data fail to eliminate.

We also attempt a direct numerical calculation in an extremely small finite-width network, rather than first measuring the answer with SMC. A neural network Gaussian process (NNGP) approximation treats the joint outputs of a random network as Gaussian and retains only their mean and covariance. It recovers most of the function ordering but severely misestimates some very-low-probability events. We then retain the non-Gaussian distribution produced by summing a finite number of hidden units, concentrate numerical samples near the target, and remove that artificial sampling bias with probability weights. When the effective number of weighted samples is sufficient, the calculation reproduces the SMC result. Function probabilities calculated from the network equations, rather than measured from training, then predict hundreds of thousands of independent AdamW runs. They nearly match the SMC probability estimates and provide a small but stable predictive improvement over the pure NNGP approximation.

The significance of this result is not that the small-network calculation is intrinsically complicated. It is that at least one finite neural network exists for which, once the architecture and parameter distribution are given, function probabilities can be calculated without observing actual training outcomes and can then predict the distribution of those outcomes. The calculation remains a numerical approximation rather than a closed-form solution, and it does not solve the scaling problem for large networks.

The same method can compare candidate labels for unseen examples. E26 selects a set of MNIST images of zeros and ones, reveals only one label, and states that the two classes must occur in a 5:5 ratio. Before the remaining labels are revealed, the low-loss parameter probability of every candidate partition is calculated. The natural zero/one partition ranks poorly when relatively high loss is allowed and becomes the most probable partition only at lower loss. Thus, given one true label, which mainly removes the otherwise valid global label-swap symmetry, and the class ratio, the natural labels of all ten images can be inferred in the low-loss regime. When the 5:5 constraint is removed, however, a candidate that assigns every image to the same class has greater parameter mass. Sparse labels and a known class ratio, or other explicit side information, can therefore be combined in prediction, and that extra information can improve the result substantially.

6. Static Distributions Differ from Adam Outcomes

When parameters are drawn without training and conditioned only on loss, we can measure the fraction assigned to every function. Actual training asks a different question: after fixing Adam or SGD and all associated training settings, repeatedly initialize the network at random, train it, and count how often each complete function appears. The latter distribution also depends on gradient directions, learning rate, momentum, the existence of accessible paths through parameter space, and the preceding training history. The two distributions can be related, but there is no reason to assume in advance that they are identical.

E17 observes both agreement and disagreement within the same small task. At relatively high allowed loss, high-probability functions in the static distribution cover most outcomes produced by Adam, SGD, and Momentum. Parameter probability therefore identifies many of the candidates that actual training is likely to encounter. At very low loss, however, Adam's most frequent function can differ from the most frequent function under direct parameter sampling. Continuing training from parameters found by SMC also produces different final function proportions from training that begins at ordinary random initialization. Parameter mass is informative, but it does not by itself determine the output of a training algorithm.

E16 separates three further questions: whether correct parameters exist, whether a network can preserve or recover the target once it is near those parameters, and whether ordinary random initialization can reach them. When only the final output of a high-dimensional parity task is supervised, ordinary initialization fails to recover the target. Intermediate supervision guides some networks near correct parameters. After all intermediate supervision is removed and only the original final loss remains, these networks retain the correct function; after a strong perturbation makes them temporarily incorrect, most recover it. Intermediate supervision therefore mainly helps training find the correct region. The decoupling and training-scaffold experiments in the earlier paper, which likewise provide intermediate targets, are better evidence about optimization accessibility than about static parameter probability.

A 10-bit input has 1,024 possible states. We first train the network on all 1,024 input-output pairs to confirm that it can learn the complete one-step Rule110 update exactly. We then provide only a subset of examples and test whether the network predicts the remaining states correctly. As the training set grows from 128 to 256 examples, the probability of recovering complete Rule110 rises rapidly. Each group of networks usually answers all training examples correctly early, but with fewer training examples it takes longer to recover the rule on the remaining states. We also perform a measurement independent of Adam's trajectory: search parameter space directly for networks with comparably low training loss and test whether they implement complete Rule110. With only 128 or 160 training examples, many such low-loss parameters already implement the full rule, while Adam starting from ordinary random initialization finds none within the finite training budget. Correct parameters therefore exist, but Adam has not yet reached them.

Changing the starting point makes the distinction sharper. With the same $n = 128$ training set and nearly the same final loss, starting from random weights, weights that have just learned Rule110, or weights at which Rule110 is already highly stable produces very different function proportions. Randomly initialized networks have not stopped moving: they switch among incorrect functions tens of thousands of times, but do not become Rule110. Starting instead from parameters on which Adam has stalled, HMC can relax and then retighten the low-loss constraint using training loss alone, without using Rule110 labels on any unseen input. This moves many parameters into the Rule110 region. After subsequent Adam fine-tuning, most remain correct. This is not yet a practical training method, but it shows that parameter sampling can reveal cases in which many correct weights exist but ordinary training has not

found them. It can also guide a parameter-remixing intervention that improves the eventual training result.

7. Beyond Truth Tables: MNIST, Kernels, and Regularization

E25 applies the method to downsampled MNIST. We first sample many networks with low training loss and then let them vote on unseen images. As the loss threshold decreases, the negative log-likelihood (NLL) of the static prediction first improves and then worsens. Its minimum occurs near the training-loss range at which Adam's validation NLL is best, so the static measurement approximately predicts Adam's best stage. Beyond this point, networks can become more mutually consistent while also becoming more confident on a small number of incorrect images. Accuracy, NLL, and inter-model agreement therefore need not be optimized at the same loss.

E25 uses only a small number of images and a binary task. To test whether parameter sampling remains useful beyond that setting, E28 runs whole-network HMC on 50,000 MNIST training images with a 4,266-parameter CNN. An ensemble formed from 480 posterior parameter settings reaches 99.03% test accuracy, comparable to repeated Adam training of the same architecture. This shows that parameter sampling does not immediately fail on a real ten-class task, but it does not show that HMC outperforms Adam. Predictions from the parameter samples agree strongly on individual images, yet the 480 parameter settings still produce 480 distinct complete test-set answer vectors, and some images are classified incorrectly by every sample. High agreement among models therefore usually indicates greater reliability, but it cannot guarantee correctness.

This observation motivates an exploratory conjecture. Functions with high agreement under a training set may be closer both to structures the current architecture expresses easily and to structures humans can describe. High agreement does not logically imply a simple function, and even a simple function need not be simple to a human. The conjecture is instead that high agreement may indicate a clear, shared, compressible structure in the training set. If that structure also admits a human-readable rule, then its compressibility may be objective rather than specific to either neural networks or humans: any search for a lower-Kolmogorov-complexity representation may encounter it. In E18 and E21, some final functions selected by many independent initializations can be summarized by low-order threshold rules. This is only a preliminary test. The conjecture is not part of the paper's core conclusions.

To ask whether E28's high accuracy actually requires finite-width parameter sampling, we add kernel baselines on an independent 8k split. An NNGP-style kernel constructed only from the covariance of randomly initialized CNN outputs matches the accuracy of finite-width HMC, while an empirical NTK that approximates training by a fixed linear model near initialization performs worse. E28 therefore shows that whole-network parameter sampling can produce high-quality predictions, but it does not show that this quality depends on finite-width information unavailable to an NNGP. On the other hand, in small Boolean tasks that can be enumerated completely, the non-Gaussian probabilities of the finite-width network do provide a small amount of predictive information beyond the NNGP. Together, these results support only a limited conclusion: finite-width information is useful in some tasks, but is not guaranteed to improve every natural-image task. One reason is simply that NNGP itself can already predict some tasks very well.

Two further groups of experiments examine the relation between weight decay and grokking. E15 and E27 show that grokking can occur without weight decay. E30 asks whether explicit L2 changes the parameter probabilities of different functions. If L2 is written directly into

the training objective,

$$J_\lambda(\theta) = L_D(\theta) + \lambda\|\theta\|^2/2,$$

then, like any other loss term, it changes which parameters satisfy a threshold on total loss. E30 compares parameter distributions in a controlled AND task at similar data-fitting error but different L2 coefficients. AND is already the most common function without L2; adding L2 further increases the fraction of parameters that implement the complete AND rule. Explicit L2 can therefore make a rule function more common by reshaping the total loss, but neither rule learning nor grokking requires weight decay.

Taken together, the experiments give a continuous account of several common training phenomena. With sufficient data, the target becomes more common than incorrect functions before loss has fallen very far, so training and validation performance improve together. Near the amount of data required to identify the target, many functions remain after hard fit, and more training runs reach the same target only as loss continues to decrease; this appears as grokking. With insufficient data, functions that exploit gaps in the training set but disagree with the external generating rule can persist and can even become more common. We call them "incorrect functions" only because they disagree with the target or test distribution specified by the experimenter. For the optimization problem actually presented to the neural network, the functions that remain dominant are not merely valid candidates but can be the better candidates: they satisfy every training example while being easier for the current architecture to realize at the relevant loss precision, or while occupying greater parameter probability. By selecting them, the network is not betraying its own objective; it is faithfully reducing the training loss it was given. What we call "overfitting" arises because the external standard by which a researcher evaluates generalization is not identical to the information available to the network. With noisy labels, a network may first exploit structure shared across samples, then memorize each incorrect label separately in order to reduce loss further, thereby overfitting. This account does not require a hidden objective inside the network that actively pursues generalization. Early stopping is an intervention by the researcher, who selects a stage with better validation performance; the preference for generalization comes from outside the training objective.

8. A Minimal Formal Framework and Its Statistical-Physics Representation

8.1 The Core Idea Without Formalism

This paper separates two observations. First, under several protocols measured here, random initialization induces a highly nonuniform function distribution, with some reusable and structurally simple functions occurring more often. Second, within prespecified function pairs and finite loss windows, lowering the loss changes the relative parameter mass of different functions. In many experiments, implementations with more shared computation retain more mass, but this direction is not a set-theoretic necessity and is not guaranteed for every architecture, loss, or measure.

From a training-loss-centered perspective, lowering loss can therefore have at least two effects: it removes functions that genuinely violate the training constraints, and it redistributes parameter mass among functions that remain feasible. The second effect depends on the joint output distribution and the particular loss slice; it cannot be identified directly with classical MDL or a theorem that the network must find the shortest program. We use **compression is intelligence** as an interpretive summary suggested by these measurements, not as a theorem that holds for every protocol.

8.2 Four Probabilities That Must Not Be Confused

First fix the network architecture, parameterization, data encoding, and per-sample loss. We must also specify two parameter distributions separately. The reference measure μ_{ref} is used to calculate parameter probabilities, while μ_{init} is the actual random initialization used by Adam or SGD. They may be the same, but they are conceptually distinct. Let the network parameters be θ, the network's class labels over the complete input domain be h_θ, and its average loss on training set D be $L_D(\theta)$.

The preceding sections use four different probabilities. The first is the probability that initialization produces function f:

$$P_{\text{init}}(f) = \Pr_{\theta \sim \mu_{\text{init}}}[h_\theta = f].$$

The second temporarily supplies the answers of target f over the entire input domain and measures the probability that random parameters realize this complete target with loss at most ϵ:

$$V^{\text{full}}(f;\epsilon) = \Pr_{\theta \sim \mu_{\text{ref}}}[L_{D_{\text{full}}(f)}(\theta) \le \epsilon].$$

Neural Kolmogorov complexity is the curve traced by $-\log_2 V^{\text{full}}$ as ϵ changes. The third probability supplies only a finite training set D and, among all parameters satisfying the loss condition, measures the fraction whose complete function is f:

$$Q^{\text{static}}_{D,\epsilon}(f) = \Pr_{\theta \sim \mu_{\text{ref}}}[h_\theta = f \mid L_D(\theta) \le \epsilon].$$

This probability includes every function that fits the training set equally well but may be wrong on unseen inputs. The fourth probability describes actual training. Fix Adam or SGD and its full training protocol, repeatedly initialize from μ_{init}, and measure how often function f appears after t steps:

$$Q^{\text{opt}}_{D,t}(f) = \Pr_{\theta_0 \sim \mu_{\text{init}}}[h_{T_{D,t}(\theta_0)} = f].$$

Here $T_{D,t}(\theta_0)$ means only "the parameters obtained after training for t steps on D, starting from θ_0." These four probabilities can be correlated, but none is equal to another by default. Initialization probability describes the preference before training. Complete-target probability describes how difficult the target itself is to realize. The finite-training-set conditional probability includes competition from every incorrect function. Actual training frequency additionally depends on the optimizer and training time.

8.3 Loss Thresholds, Partition Functions, and Function Competition

Statistical physics provides two ways to organize these parameters. The first, used in most of our experiments, imposes a hard threshold and retains only parameters for which $L_D(\theta) \le \epsilon$. For a fixed-D function cell f, let $\rho_{\Pi,U,D,f}(E)$ denote the parameter-probability density near loss E that also realizes f. The cumulative mass of that cell below the threshold is

$$\Omega_{\Pi,U,D}(f;\epsilon) = \int_0^\epsilon \rho_{\Pi,U,D,f}(E)\, dE.$$

Different functions have different $\rho_{\Pi,U,D,f}(E)$, so their cumulative masses can decrease at different rates as ϵ is lowered, and their curves can cross. This is a statistical-physics representation of the fixed-D function-mass profile; the full-target Neural K-profile remains the $-\log_2 V^{\text{full}}$ object defined above.

The second method does not remove high-loss parameters. Instead, it assigns greater weight to lower-loss parameters:

$$Z_{D,f}(\beta) = \int \mathbf{1}[h_\theta = f] e^{-\beta L_D(\theta)}\, d\mu_{\text{ref}}(\theta), \qquad Z_D(\beta) = \sum_f Z_{D,f}(\beta).$$

Z_D is the partition function, the total weighted mass of all parameters. Function f has probability $Z_{D,f}/Z_D$ under this distribution. As β increases, low-loss parameters receive greater relative weight. If L_D in the implementation is the mean negative log-likelihood over n examples, the standard Bayesian posterior corresponds to $\beta = n$; if total loss is used, it corresponds to $\beta = 1$.

Statistical physics commonly calls $-\beta^{-1} \log Z$ the free energy. It reflects both how low the loss is and how many parameters achieve that loss. One function may attain a lower minimum loss but be realized by very few parameters, while another may have slightly higher loss but much greater parameter mass. As ϵ decreases or β increases, the balance between these factors changes, so the most common function can change as well. The stage transition in E14 and the curve crossing in E24 can both be written as competition among the $Z_{D,f}$ or $V_{D,f}$ of different functions.

Hard-threshold statistics and $e^{-\beta L}$ weighting are derived from the same loss distribution, but they are not the same probability. SMC primarily estimates the former; HMC can target the latter when the Gibbs target is explicitly specified. Calling loss "energy" does not make Adam or SGD sample automatically from either static distribution.

8.4 Information From a New Example and the Order of Examples

The same static parameter probabilities can describe how many candidate parameters a new example eliminates, but hard constraints and loss weighting must be defined separately.

First consider the hard-constraint version. For a set of observed examples S, let $A(S)$ be the set of parameters that answer all of them correctly, and define

$$Z_{\text{hard}}(S) = \mu(A(S)), \qquad C_{\text{hard}}(S) = -\log_2 Z_{\text{hard}}(S).$$

After adding a new example i, the increase in cost is

$$\Delta I_{\text{hard}}(i \mid S) = -\log_2 \frac{Z_{\text{hard}}(S \cup \{i\})}{Z_{\text{hard}}(S)}.$$

The ratio has a direct conditional-probability interpretation: among parameters that already answer every example in S correctly, what fraction also answers the new example i correctly? If the new example eliminates almost no parameters, the information increment is small. If it eliminates most existing candidates, the increment is large.

This hard information increment describes conditioning inside a fixed parent event. If, after adding the example, we instead redefine the same average-loss threshold through $\{L_{S\cup\{i\}} \le \epsilon\}$, the new event can absorb parameters outside the old parent event and can also remove parameters inside it. It is therefore a different event and cannot be assigned the same conditional-probability interpretation.

The loss-weighted version uses a different partition function. Let the loss on one example be $\ell_i(\theta)$ and the total loss over S be

$$E_S(\theta) = \sum_{j \in S} \ell_j(\theta).$$

Define

$$Z_\beta(S) = \int e^{-\beta E_S(\theta)}\, d\mu(\theta), \qquad C_\beta(S) = -\log_2 Z_\beta(S).$$

The information increment can again be defined as the difference between adjacent costs, but the ratio no longer means "the fraction of parameters that answer the new example correctly." It measures how much the new example reduces parameter weights on average under the current Gibbs-weighted distribution. The hard-constraint version records whether a parameter passes a threshold; the Gibbs version records the continuous penalty that the new example assigns to each parameter.

For either definition, as long as the same Z is used throughout, sequentially adding all examples in the complete training set D gives

$$\sum_{t=1}^{n} \Delta I(\pi_t \mid S_{t-1}) = C(D) - C(\varnothing).$$

Each information increment still depends on sample order. An example added early may eliminate many candidates, whereas the same example added late may contribute little because previous examples have already removed them. The sum of all increments nevertheless depends only on the common starting and ending points, since all intermediate terms cancel when adjacent cost differences are added.

E22 checks both the hard-constraint and Gibbs-weighted versions in a 3-bit task. It enumerates 6,561 partially labeled states, 256 complete rules, and all 40,320 orders for each rule. Under both definitions, the cumulative cost satisfies the identity to numerical precision. The experiment verifies that the probabilities used by the implementation close consistently and measures the endpoint costs of different complete rules. Order independence itself remains an ordinary probability-chain identity, not a new physical law. If one wants to assign each example an average contribution independent of its position, its information increment can be averaged over all insertion orders, yielding a Shapley-style allocation. The order independence here belongs only to the static probability calculation; it does not imply that SGD trained with different minibatch orders must produce the same result.

8.5 What Is Determined by Data, Network, Loss, and Optimizer?

Factor	Primary role
Data	Specify the answers that must be satisfied, leave some incorrect functions uneliminated, and determine how many candidates each new example removes
Network architecture, encoding, and parameter-sampling measure	Determine which intermediate computations can be reused easily and how much parameter probability is associated with each function

Factor	Primary role
Loss threshold or β	Determine the required precision of training fit and the relative weight assigned to low-loss parameters
Optimizer and training history	Determine the path followed from initialization and which parameters are actually reached in finite time

This division of roles neither requires static parameter probabilities to equal Adam outcomes nor permits every predictive failure to be dismissed by saying "the optimizer cannot find it." Accessibility must be tested concretely, for example by changing the starting point, adding or removing intermediate supervision, extending training time, or remixing parameters.

9. Relation to Previous Work

We do not claim to have introduced simplicity bias, parameter volume, free energy, MDL, or grokking. Previous literature also calls the collection of all models satisfying the training constraints a version space. Work on parameter-to-function priors has shown that random networks generate some functions far more often than others (Dingle et al., 2018; Valle-Perez et al., 2019; Mingard et al., 2021, 2025). Our addition is to measure the continuous loss dimension: functions on unseen inputs continue to change after every training example is classified correctly, and even in the absence of actual training, the parameter probabilities of different functions shrink at different rates. "Initialization preference plus one-time conditioning on the training labels" is therefore only the coarsest approximation.

The statistical physics of learning has long studied the size of the parameter set satisfying all training constraints, how weighting parameters by $e^{-\beta L}$ changes the distribution as data are added, and how many examples generated by a known rule are needed for a learning model to recover that rule (Gardner, 1988; Levin, Tishby and Solla, 1989; Watkin, Rau and Biehl, 1993). Work on flat minima, local entropy, and dense solution clusters has likewise related the width or probability of low-loss parameter regions to generalization (Hochreiter and Schmidhuber, 1997; Baldassi et al., 2015; Chaudhari et al., 2017).

Our framework adds a function-level partition to these approaches. Parameters are first grouped by the class labels the network assigns to every possible input. When these labels agree, their logits can be compared further. We then observe how the probability of each group changes with the loss threshold and where two curves cross. These parameter probabilities still depend on the parameter coordinates and sampling measure, so the reparameterization critique of Dinh et al. (2017) applies here as well.

The two Neural K profiles in this paper share with Solomonoff induction, Kolmogorov complexity, and MDL the idea that negative log probability can be interpreted as description length (Solomonoff, 1964; Kolmogorov, 1965; Grunwald, 2004; Blier and Ollivier, 2018). Algorithmic-statistics structure functions also represent allowed error and description length as a curve (Gacs, Tromp and Vitanyi, 2001). The paper's $K^{N,\mathrm{full}}$ is the complete-target profile, while $K^{N,\mathrm{FD}}$ is the fixed-training-set function-cell profile; both are computable code lengths induced by a concrete network setup, not machine-independent $K_U(f)$. They also differ from PAC-Bayes, which typically combines training error with the divergence between a post-training parameter distribution and a prior to bound generalization error (McAllester, 1999). In singular learning theory, the real log canonical threshold (RLCT) describes how total probability scales asymptotically with sample size, while the local learning coefficient

estimates a related exponent near a particular network solution (Watanabe, 2009; Lau et al., 2025). These quantities may help explain the local shape of the extreme low-loss tail, but they are not currently identical to our global, finite-precision curves resolved by complete function.

The grokking literature has explained delayed generalization in terms of internal circuit formation, regularization, data quantity, continued motion after training loss reaches zero, optimizer instability without explicit L2, directions unconstrained by the data, and slow dynamics analogous to glassy systems (Power et al., 2022; Nanda et al., 2023; Zhang et al., 2025). We add an observation at the level of complete functions: when the training set is first classified correctly, different initializations do not share one incorrect function; as the amount of data grows, the target becomes common at progressively higher loss thresholds. This connects three regimes. With enough data, training and validation improve together. Near the boundary, generalization is delayed. With insufficient data, many distinct functions remain for a long time.

NNGP, NTK, and Bayesian HMC are important computational baselines for this work (Lee et al., 2018; Jacot, Gabriel and Hongler, 2018; Izmailov et al., 2021). They test how much can be explained respectively by the covariance of outputs at initialization, by a linearized approximation around initialization, or by sampling the complete parameter distribution. The purpose of these comparisons is not to claim that one method is always best, but to identify the tasks in which complete finite-width parameter probabilities provide additional information.

10. Limitations and Conclusion

First, every probability in this paper depends on the experimental setup. Changing the initialization distribution, network scaling, parameter coordinates, or parameter-sampling measure can change the function ranking. This dependence cannot be used to evade counterexamples. Instead, future work should predict, from concrete structures such as convolution, residual connections, attention, and weight sharing, which functions retain greater parameter probability as the loss threshold decreases.

Second, extremely low-probability regions are difficult to sample reliably. As SMC repeatedly removes worse parameters and duplicates better ones, its final population may descend from only a few early particles and miss a disconnected low-loss region. A high local acceptance rate in HMC likewise does not show that the sampler moves between disconnected regions. Credible evidence should therefore include agreement between ordinary random sampling and rare-event methods in their overlap region, consistency across independent sampling runs and paired experiments, closure under ordinary conditional-probability identities, and prospective predictions that succeed in new experiments. It should not rest on one enormous probability ratio alone.

Third, the network's answer on every possible input can be enumerated only when the input domain is finite and small. Even then, discrete answers omit margins, probability calibration, and internal representations. Continuous inputs, regression, language, and generative models require another measurable description of behavior, such as predictions on a set of important queries or responses to controlled perturbations. The 50k MNIST experiment shows that a parameter ensemble obtained without ordinary gradient training can still make high-quality predictions beyond Boolean truth tables. It does not show that complete-function probabilities are computationally accessible in large models.

Fourth, curves obtained by direct parameter sampling cannot completely predict actual training. The MUX experiment shows that knowing the low-loss volume of a complete target

does not determine its sample requirement, because a finite training set may fail to eliminate its strongest competitors. The Rule110 experiment shows that Adam may fail to find correct parameters within finite time even when those parameters are abundant. The NNGP baseline shows that, on some natural-image tasks, covariance between random-network outputs already gives strong predictions. Our account must accommodate these counterexamples and continue to make prospective predictions from within those constraints.

The account is directly testable and can fail. The following findings would weaken its core:

1. Another reliable probability-estimation method fails to reproduce the changing parameter probabilities or curve crossings between functions.
2. Neural K curves fixed in advance consistently fail across new tasks to predict unseen labels, sample requirements, or function frequencies after repeated training.
3. The probabilities of complete targets and finite training sets fail to satisfy ordinary conditional-probability identities.
4. Changes predicted by direct parameter sampling remain unrelated to the outcomes of data interventions or parameter-remixing interventions.

Other findings would only limit the scope of the framework. If changing the network changes the function ranking, then the result is architecture dependent. If NNGP fully explains a natural task, then finite-width parameter probability is unnecessary for that task. Neither result by itself contradicts the observation that functions continue to change after the training set has been classified correctly.

This study begins with a simple phenomenon: why do networks capable of memorizing their training sets so often give structured, generalizable answers beyond them? The experiments first confirm that random initialization already makes some functions easier to produce than others. They then show that training does more than remove functions that answer the training labels incorrectly. After hard fit, answers on unseen inputs continue to change. Even without actual training, lowering the loss threshold makes the parameter probabilities of different functions decrease at different rates. The Neural Kolmogorov complexity curve turns this last observation into a measurable probability profile and uses it to predict sample requirements, unseen labels, and the function frequencies produced by repeated training.

These experiments do not prove that neural networks always search for the shortest program defined by humans. Their direct conclusion is narrower: for a fixed network, encoding, and parameter-sampling measure, the parameter probabilities associated with different functions change differently as the loss threshold decreases. Reusing a shared computation reduces independent parameter adjustment and is one experimentally supported explanation, but we do not yet have a complete quantitative formula. Data determine which incorrect answers remain uneliminated. Network structure determines which computations are easy to implement. The loss threshold determines the required output precision. The optimizer determines which parameters training actually finds. The main contribution of this paper is to turn the claim that "networks seem to prefer simple rules" into an experimental question that can be measured, used for prospective prediction, and tested against counterexamples.

Experimental Materials

E01 through E30 are stable experiment identifiers in the public evidence package. Each identifier links to an experimental objective, a frozen reproduction script, a result report, and an explicit boundary on the supported conclusion. The current experiment index is available in the public GitHub repository, and the project archive is available at Zenodo DOI

10.5281/zenodo.20446430. If a concise description in the main text is ambiguous, the protocol and conclusion boundary in the corresponding experiment record take precedence.

References

Baldassi, C., Borgs, C., Chayes, J. T., Ingrosso, A., Lucibello, C., Saglietti, L., and Zecchina, R. (2015). Subdominant Dense Clusters Allow for Simple Learning and High Computational Performance in Neural Networks with Discrete Synapses. *Physical Review Letters*, 115, 128101.

Blier, L., and Ollivier, Y. (2018). The Description Length of Deep Learning Models. *NeurIPS 2018*.

Chaudhari, P., Choromanska, A., Soatto, S., LeCun, Y., Baldassi, C., Borgs, C., Chayes, J., Sagun, L., and Zecchina, R. (2017). Entropy-SGD: Biasing Gradient Descent Into Wide Valleys. *ICLR 2017*.

Dingle, K., Camargo, C. Q., and Louis, A. A. (2018). Input-Output Maps Are Strongly Biased Towards Simple Outputs. *Nature Communications*, 9, 761.

Dinh, L., Pascanu, R., Bengio, S., and Bengio, Y. (2017). Sharp Minima Can Generalize for Deep Nets. *ICML 2017*, PMLR 70:1019-1028.

Gacs, P., Tromp, J., and Vitanyi, P. M. B. (2001). Algorithmic Statistics. *IEEE Transactions on Information Theory*, 47(6):2443-2463.

Gardner, E. (1988). The Space of Interactions in Neural Network Models. *Journal of Physics A: Mathematical and General*, 21(1):257-270.

Grunwald, P. (2004). A Tutorial Introduction to the Minimum Description Length Principle. arXiv:math/0406077.

Hochreiter, S., and Schmidhuber, J. (1997). Flat Minima. *Neural Computation*, 9(1):1-42.

Izmailov, P., Vikram, S., Hoffman, M. D., and Wilson, A. G. (2021). What Are Bayesian Neural Network Posteriors Really Like?. *ICML 2021*, PMLR 139:4629-4640.

Jacot, A., Gabriel, F., and Hongler, C. (2018). Neural Tangent Kernel: Convergence and Generalization in Neural Networks. *NeurIPS 2018*.

Kolmogorov, A. N. (1965). Three Approaches to the Quantitative Definition of Information. *Problems of Information Transmission*, 1(1):1-7.

Lau, E., Furman, Z., Wang, G., Murfet, D., and Wei, S. (2025). The Local Learning Coefficient: A Singularity-Aware Complexity Measure. *AISTATS 2025*, PMLR 258:244-252.

Lee, J., Bahri, Y., Novak, R., Schoenholz, S. S., Pennington, J., and Sohl-Dickstein, J. (2018). Deep Neural Networks as Gaussian Processes. *ICLR 2018*.

Levin, E., Tishby, N., and Solla, S. A. (1989). A Statistical Approach to Learning and Generalization in Layered Neural Networks. *COLT 1989*, pp. 245-260.

McAllester, D. A. (1999). PAC-Bayesian Model Averaging. *COLT 1999*.

Mingard, C., Valle-Perez, G., Skalse, J., and Louis, A. A. (2021). Is SGD a Bayesian Sampler? Well, Almost. *Journal of Machine Learning Research*, 22(79):1-64.

Mingard, C., Rees, H., Valle-Perez, G., and Louis, A. A. (2025). Deep Neural Networks Have an Inbuilt Occam's Razor. *Nature Communications*, 16, 220.

Nanda, N., Chan, L., Lieberum, T., Smith, J., and Steinhardt, J. (2023). Progress Measures for Grokking via Mechanistic Interpretability. arXiv:2301.05217.

Power, A., Burda, Y., Edwards, H., Babuschkin, I., and Misra, V. (2022). Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets. arXiv:2201.02177.

Solomonoff, R. J. (1964). A Formal Theory of Inductive Inference, Parts I and II.

Information and Control, 7.

Soudry, D., Hoffer, E., Nacson, M. S., Gunasekar, S., and Srebro, N. (2018). The Implicit Bias of Gradient Descent on Separable Data. *Journal of Machine Learning Research*, 19(70):1-57.

Valle-Perez, G., Camargo, C. Q., and Louis, A. A. (2019). Deep Learning Generalizes Because the Parameter-Function Map Is Biased Towards Simple Functions. *ICLR 2019*.

Watkin, T. L. H., Rau, A., and Biehl, M. (1993). The Statistical Mechanics of Learning a Rule. *Reviews of Modern Physics*, 65:499-556.

Watanabe, S. (2009). Algebraic Geometry and Statistical Learning Theory. Cambridge University Press.

Wilson, A. G. (2025). Position: Deep Learning Is Not So Mysterious or Different. *ICML 2025*, PMLR 267:82326-82346.

Zhang, X., Shang, Y., Yang, E., and Zhang, G. (2025). Is Grokking a Computational Glass Relaxation?. *NeurIPS 2025*.